Machine Learning in Finance
Project Options and Instructions

Karen Bozanian

February 2026

Overview

This document describes the three projects you may choose from for the final group assignment (groups of 4 students; duration: 4 weeks). Each group chooses exactly one project. The projects are designed to reflect the course's twin pillars: quantitative finance (asset management, derivatives pricing, high-frequency trading) and machine learning (building from scratch, reproducing research, and applying existing models to challenges).

If you want me to adapt or clarify any project description, tell me which one and I will update the brief.

1. Logistics and Deadlines

- Group size: 4 students.
- Due date: 28/03
- For every project the required deliverables are:
  1. A written report (PDF, word)
  2. Reproducible code (well-documented, runnable). Preferably a python notebook, or R-code
  3. Presentation slides for the oral presentation

2. Grading

- Final report (technical quality, clarity)
- Code quality and reproducibility (runnable, documented, tests)
- Presentation and demo (clarity, communication, defense)
- Individual contribution

3. Project 1: Implement an Observation-Driven HMM from Scratch

Goal: Implement an observation-driven Hidden Markov Model (HMM) tailored to financial time series and provide full mathematical proofs of the recursions and algorithms used.

We studied the Hidden Markov Model on the first two sessions of our class.
The goal here is to adapt all the recursions and formulas (likelihood, forward algorithm, backward

algorithm, gamma and xi definition, Viterbi algorithm and finally the EM algorithm) to the new model with the change of hidden process dynamic.

The ODHMM is the same as the HMM but has the following hypothesis on the hidden process instead : the transitions of the hidden process are not markovian, but they additionnally depend on the last observation :

$$p(X_{t+1} = j \mid X_t = i) \to p(X_{t+1} = j \mid X_t = i, Y_t = y) \equiv q_{ij}^y$$

For the theoritical part, we will consider the general formulation $p(X_{t+1} = j \mid X_t = i, Y_t = y) \equiv q_{ij}^y$, while for the numerical experiment, we will choose the following parametrization:

$$p(X_{t+1} = j \mid X_t = i, Y_t = y) = \frac{q_{ij} b_j(y)}{\sum\limits_{j' \in E} q_{ij'} b_{j'}(y)}$$

which represents a Markov chain contaminated by the process Y. Recall $y \mapsto b_j(y)$ is the probability density function in state $j \in E$ which can be chosen gaussian.

The final goal is to consider the following strategy : everyday, an agent can choose his exposure at rate u_t on a risky asset and $1 - u_t$ on a risk-free asset of rate r. The dynamic of the wealth of the portfolio is given by :

$$V_{t+1} = V_t(1 + (1 - u_t)r + u_t Y_{t+1})$$

We will model the returns $(Y_t)_t$ of a "risky" asset (your choice : S&P500, CAC40, EUROSTOXX600,...) by a ODHMM and try to solve the following allocation problem :

$$\max_u \mathbb{E}\left[\mathcal{U}(V_{t+1}^u) \mid Y_1^t\right]$$

which is the one-day ahead optimal allocation. We can show that

$$\begin{aligned}
\mathbb{E}\left[\mathcal{U}(V_{t+1}^u) \mid Y_1^t\right] &= \mathbb{E}\left[\mathcal{U}(V_t(1 + (1-u)r + uY_{t+1})) \mid Y_1^t\right] \\
&= \int_{\mathbb{R}} \mathcal{U}(V_t(1 + (1-u)r + uy)) f_{Y_{t+1}|Y_1^t}(y) dy \\
&= \int_{\mathbb{R}} \mathcal{U}(V_t(1 + (1-u)r + uy)) \frac{1}{\mathcal{L}(Y_1^t)} \sum_{x_t \in E} \sum_{x_{t+1} \in E} b_{x_{t+1}}(y) u_t(x_t) q_{x_t, x_{t+1}}^{y_t} dy \\
&= \int_{\mathbb{R}} \mathcal{U}(V_t(1 + (1-u)r + uy)) \sum_{x_t \in E} \sum_{x_{t+1} \in E} b_{x_{t+1}}(y) \gamma_t(x_t) q_{x_t, x_{t+1}}^{y_t} dy \\
&= \int_{\mathbb{R}} \sum_{x_t \in E} \sum_{x_{t+1} \in E} \mathcal{U}(V_t(1 + (1-u)r + uy)) b_{x_{t+1}}(y) \gamma_t(x_t) q_{x_t, x_{t+1}}^{y_t} dy
\end{aligned}$$

The project should present some graphs showing the result of viterbi and gamma decoding on toy/simulated data, and the evolution of the wealth of V_t in comparision to the risky asset.

4. Project 2: Replication of a Deep Reinforcement / Deep Learning Paper (Deep Optimal Stopping OR Deep Hedging)

Goal: Reproduce the main results of either the *Deep Optimal Stopping* or the *Deep Hedging* paper and perform careful analysis, ablations, or extensions.

Deep Optimal Stopping

You will reproduce and apply the deep optimal-stopping idea: represent an optimal stopping time as a sequence of 0/1 stopping decisions and learn those decisions with neural networks. The goal is to compare this learned policy to the classical Longstaff–Schwartz least-squares Monte-Carlo (LSM) method on typical Bermudan/American payoffs (the paper's canonical example is a Bermudan max-call). The project should show whether the learned stopping rule gives competitive prices, how tight the primal/dual bounds are, and how cost/time tradeoffs behave.

First implement the market simulator and payoffs (start with a multidimensional Black–Scholes; the paper also tests a discretized fractional Brownian example). Implement LSM as the classical baseline and validate it on small cases. Implement the deep stopping pipeline that the paper describes: at each exercise date train a neural network F^θ to produce a soft stopping score, then convert it to a hard decision $f^\theta = 1_{[0,\infty)} \circ (\cdots)$; the paper trains networks with standard deep-learning practices (Xavier initialization, batch normalization, Adam). In the authors' examples they used networks of moderate depth (the paper shows examples with depth $I = 3$ and hidden widths like $q1 = q2 = d + 40$, trained with mini-batch gradient ascent. After training, evaluate the learned policy on fresh Monte-Carlo paths to obtain a primal lower bound. Implement the dual (upper bound) estimator (martingale correction / nested continuation paths) to obtain an upper bound on the true price; the paper uses large nested budgets for tight bounds. Throughout keep the simulator, seeds and test sets identical for LSM and deep stopping so comparisons are fair.

The paper reports very large numerical budgets in some examples (e.g. huge test sample sizes and large inner sample counts for the dual). Students with limited hardware should keep the same methodology but scale budgets down systematically and document the scaling. Concretely: use smaller network widths (reduce $q1, q2$ from $d+40$ to something like $d+10$ or just d while debugging), start with batch sizes the machine can handle (the authors sometimes used very large batches such as 8,192; if you have one consumer GPU, a practical batch will often be 512–4,096), and cut training steps down for early development (the paper mentions training schedules like a few thousand steps in some experiments and used larger runs elsewhere). For bounds, reduce the primal test set from millions to tens or hundreds of thousands and reduce the dual outer/inner sizes (e.g. outer K_U a few hundreds and inner continuation J a few hundreds) and then run a sensitivity study to show how increasing these budgets tightens the bound. If no GPU is available, try Google Collab.

Report the primal lower bound from the learned policy, the LSM price, and the dual upper bound (when available), together with estimated standard errors or confidence intervals. Present the bound gap $U - L$ and a table that shows how the gap shrinks as you increase inner sample sizes J (a short sensitivity study). For a two-dimensional example visualize the learned exercise region and compare it qualitatively to LSM decisions. Include training diagnostics (loss or objective vs iterations), wall-clock training time and memory usage for typical runs, and a short section on hyperparameter sensitivity (network width, batch size, number of training steps). If you downsized budgets, include explicit notes and one plot showing how estimated price or gap changes with sample size so a reader can see the scaling behavior.

Deep Hedging

The task is to implement the Deep Hedging framework and compare its performance against classical hedging strategies. In Deep Hedging, the hedger's discrete-time policy is parametrized by neural

networks that map observed market features together with the current portfolio state to the next holdings. The networks are trained to minimize a chosen risk measure (for example CVaR or an entropic/exponential objective) in the presence of realistic market frictions such as proportional transaction costs.

The comparison should be carried out in two market environments at minimum. First, use Black–Scholes (one underlying, closed-form delta hedge available) to validate correctness and to build intuition. Second, use a stochastic volatility model (Heston) to evaluate the methods in a more realistic setting where classical greeks must be computed numerically. The goal is to show where Deep Hedging provides measurable benefits (tail risk reduction, lower expected cost under constraints, different indifference prices) and to document the computational trade-offs.

Begin with the market simulator and classical baselines. Implement a discrete-time path generator for (a) Black–Scholes and (b) a Heston model (Euler or a simple scheme is acceptable if you document it; the paper used exact sampling for CIR where available). Use a time horizon and rebalancing schedule that are realistic yet manageable: the paper examples used $T = 30/365$ with daily rebalancing (30 steps), which is a good default.

For classical hedges implement:

- Black–Scholes delta hedge using the closed-form delta.
- Model delta hedge for Heston: compute greeks numerically by the method of your choice (finite differences, monte carlo,...)

Implement the Deep Hedging parametrization. At each rebalancing time (t_k) let the network (F_k^θ) produce the holdings vector $\delta_k^\theta = F_k^\theta\big(I_k, ; \delta_{k-1}^\theta\big)$, where (I_k) denotes current market features (for example spot, log-spot, recent realized/estimated vol, previous hedges, etc.) and δ_{k-1}^θ is the previous position. The paper used feed-forward networks with two hidden layers per time step; a practical architecture they report is input size $N_0 = 2d$, hidden sizes $N_1 = N_2 = d + 15$, output $N_3 = d$, with ReLU activations and batch normalization applied before activations.

Choose and implement risk measures to optimize:

- CVaR (conditional value at risk / expected shortfall) for tail-risk focused hedging.
- Entropic (exponential) risk: minimize $J(\theta) = \mathbb{E}\big[\exp(-\lambda[-Z + (\delta^\theta! \cdot S)_T - C_T(\delta^\theta)])\big]$ and recover the certainty equivalent (indifference price) from the optimized objective as described in the paper.

Include transaction costs in the model; proportional costs per trade are the simplest and align with the paper's experiments. The cost function $C_T(\delta)$ should accumulate costs of changes in holdings weighted by current mid prices.

Training: use mini-batch gradient descent (Adam). The authors report successful runs using Adam with learning rate ≈ 0.005 and batch size 256, Xavier initialization and batch normalization. Train until the empirical objective stabilizes; save checkpoints and log training diagnostics.
Evaluation: after training, evaluate all policies (deep hedge and classical hedges) on identical out-of-sample path sets to compute terminal P&L distributions, tail metrics, turnover and transaction costs, and any indifference prices if computed. Keep the simulator, seeds and test sets identical for fair comparison.

Keep the methodology the same but scale numerical parameters to your hardware: reduce hidden widths if memory is tight (use (d) or (d+5) instead of (d+15) while debugging), lower batch size (64 or 128 if 256 does not fit) and increase epochs if needed, and cut the number of gradient updates for initial experiments (start with a few thousand and increase only if training is stable). For evaluation use tens of thousands rather than hundreds of thousands of test paths if necessary, and for indifference pricing or other nested estimators limit inner sample counts and present a sensitivity study showing how estimates change as inner budgets increase. If exact Heston/CIR samplers are unavailable, state and justify the discretization used. Always use the same simulator and the same out-of-sample test sets for all compared methods and report the trade-offs you made (reduced network size, fewer training steps, smaller test ensembles) together with a short convergence/sensitivity plot that demonstrates how key metrics move with budget.

Present concise quantitative tables and a small set of illustrative figures: for each experimental configuration report mean and standard deviation of terminal P&L, VaR and CVaR (95% and 99%), average turnover and total transaction costs, and (if applicable) indifference prices; show P&L histograms or overlaid empirical CDFs to make tail differences visible. Include a plot of CVaR (or another tail metric) versus transaction cost level to demonstrate regimes where the learned hedge helps, and if you computed indifference prices show a log–log plot of $p_\varepsilon - p_0$ versus ε with an empirical slope (the literature reports a (2/3) type exponent in some settings). Provide training diagnostics (objective value versus iterations), wall-clock training time and hardware used, and one short robustness study (vary network width or batch size) so readers can see sensitivity to hyperparameters. Conclude with a brief interpretation of why the learned policy succeeded or failed in each regime.

5. Project 3: Solve a CFM Challenge Data Problem

Goal: Choose a challenge from https://challengedata.ens.fr/ (preferably given by CFM, or another public quantitative finance challenge) and produce a compelling, reproducible solution that addresses both ML performance and realistic trading/financial considerations.

Required items:

- Problem statement:
  - Define the task clearly (prediction, ranking, signal generation, hedging, etc.) and the evaluation metric used by the challenge.
  - Provide baseline(s) and justify your chosen evaluation metrics (including finance-aware metrics: P&L, Sharpe, drawdown, hit-rate, or domain-specific metrics).
- Modeling and feature engineering:
  - Describe feature construction, feature selection, and any domain-specific transformations.
  - Try multiple model classes (e.g., tree-based, neural networks, ensemble) and compare.
- Trading realism and backtesting:
  - Include realistic constraints: transaction costs, slippage, latency approximations (if relevant), position limits, and risk controls.
  - Provide a backtest engine or a careful simulation of deployed behavior.

Evaluation focus:

- Effectiveness on the challenge metric
- Soundness of experimental protocol (no look-ahead bias, proper cross-validation/backtesting).
- Thoroughness of feature engineering and model comparison
- Quality and realism of backtest and risk controls

Contact and Clarifications

If you need clarifications about project scope, allowed libraries, or dataset access, please contact me early by mail at the following address : karen.bozanian@cpram.com.
If you want me to tailor any project description (e.g., set a concrete dataset for Project 1 or pick a specific CFM challenge for Project 3), tell me which one and I will prepare a short addendum.